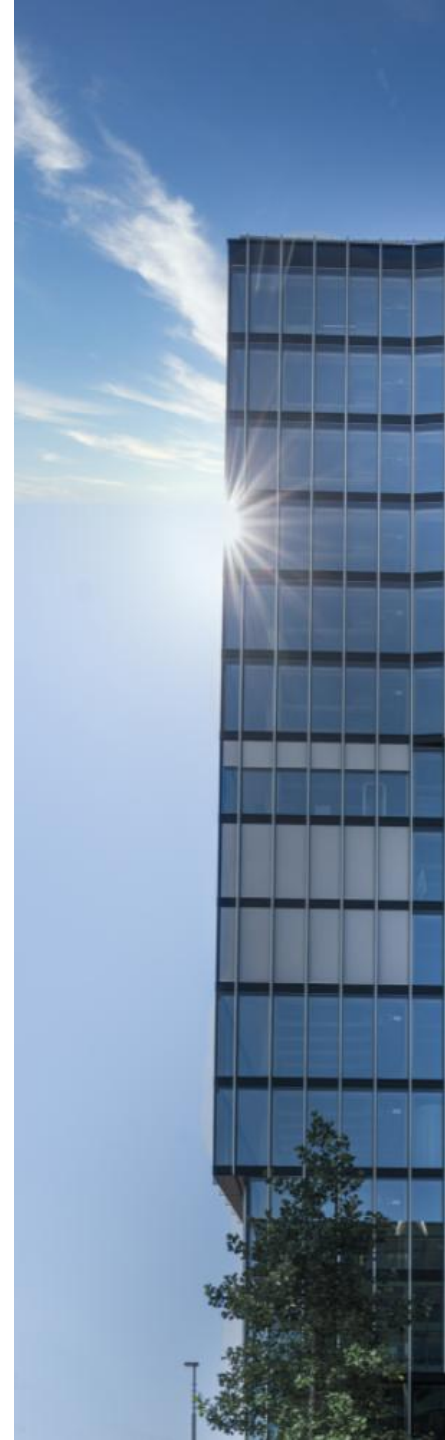


Objektorientierte Programmierung

I/O-Datenströme

Eingabe/Ausgabe in Java

Roland Gisler



Inhalt

- Konzept der I/O-Datenströme
- Byte-Datenströme
- Erweitertes Exceptionhandling
- Zeichen-Datenströme
- Encoding
- Dateihandling

Lernziele

- Sie verstehen das Konzept der I/O-Datenströme.
- Sie können zwischen Byte- und Zeichen-Datenströmen unterscheiden und kennen die Unterschiede.
- Sie wissen, was Encoding ist, und sind sich der dadurch verursachten, potenziellen Probleme bewusst.
- Sie können einfache Byte- und Datenströme in Dateien lesen und schreiben.
- Sie kennen einige fundamentale Klassen für das Dateihandling.

Konzept der I/O-Datenströme

I/O-Datenströme in Java - Grundlagen

- I/O-Datenströme (Streams) sind ein grundlegendes Konzept für die Dateneingabe und -ausgabe.
- Ein Java-Programm kann auf abstrakter Ebene:
 - Daten aus einem beliebigen Eingabestrom lesen.
 - Daten in einen beliebigen Ausgabestrom schreiben.
- Dadurch sind wir sehr flexibel, aus welcher Art von Datenquelle und Datensenke die Daten verarbeitet werden. Beispiele:
 - Eine Datei vom Dateisystem (FS- und OS-unabhängig).
 - Eingaben von Tastatur und Ausgabe auf Shell/Konsole.
 - Netzwerkkommunikation (TCP/IP-Socket etc.)
- Die wesentlichen Klassen befinden sich in den beiden Packages **java.io** (älter) und **java.nio** («new io», neuere Ergänzungen).

Datenströme – Streams und Pipes von Unix

- Das Konzept der Datenströme wurde ursprünglich für Multics/Unics → Unix erfunden, bekannte Streams sind:
 - `stdin` – Standard-Eingabe (typisch Tastatur).
 - `stdout` – Standard-Ausgabe (typisch Shell/Konsole).
 - `stderr` – Standard-Ausgabe für Fehlermeldungen.
- Unter Unix/Linux lassen sich diese Streams mit sogenannten Pipes (Queues) ad-hoc miteinander verbinden, und so die Ein- und Ausgaben von Programmen untereinander verknüpfen oder auch in Dateien umleiten.
 - Beispiel:
- Nebenbei: Im Jahr 2025 wird Unix bereits 56 Jahre alt.

https://hackaday.com/2019/11/17/unix-version-0-running-on-a-pdp-7-in-2019/?fbclid=IwAR1l3P5umt8K0M3KpB_zeZ5cEbVhfarTFXx2bdBpXpcGmwopNx7qThNvJUI

Zwei Arten von Datenströmen

Java unterscheidet zwischen zwei Arten von Datenströmen:

- **Byte**-Datenströme (Byte-Streams), reine Folge von 8-Bit-Werten:
 - Binäre Daten, von Menschen **nicht** direkt interpretierbar.

00000080:	9E D9 79 71 DA B8 17 22	DA B8 17 22 DA B8 17 22		üyqú, ."ú, ."ú, ."ú, ."ú
00000090:	FD 7E 7A 22 DB B8 17 22	FD 7E 97 22 DB B8 17 22		ý~z"ú, ."ý~"ú, ."ý~"ú, ."ý~"ú
000000A0:	C4 EA 94 22 78 B8 17 22	C4 EA 82 22 D0 B8 17 22		âê" "x, ."âê" "0, ."âê" "0, ."âê" "0
000000B0:	C4 EA 93 22 81 B8 17 22	FD 7E 6C 22 C3 B8 17 22		âê" " , ."ý~1"â, ."âê" "0, ."âê" "0
000000C0:	DA B8 16 22 A7 B9 17 22	C4 EA 9D 22 C5 B8 17 22		ú, ."ú, ."ú, ."ú, ."ú, ."ú, ."ú, ."ú
000000D0:	C4 EA 83 22 DB B8 17 22	C4 EA 86 22 DB B8 17 22		âê" "ú, ."âê" "ú, ."âê" "ú, ."âê" "ú
000000E0:	52 69 63 68 DA B8 17 22	00 00 00 00 00 00 00 00		Richú, .".....
000000F0:	00 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00		

- **Zeichen**-Datenströme, Folge von Zeichen:
 - Text-Daten, in der Regel für Menschen lesbar.

```
/*
 * Copyright 2016 Hochschule Luzern - Informatik.
 *
 * Licensed under the Apache License, Version 2.0 (the "License");
 * you may not use this file except in compliance with the License.
 * You may obtain a copy of the License at
 *
 * http://www.apache.org/licenses/LICENSE-2.0
```

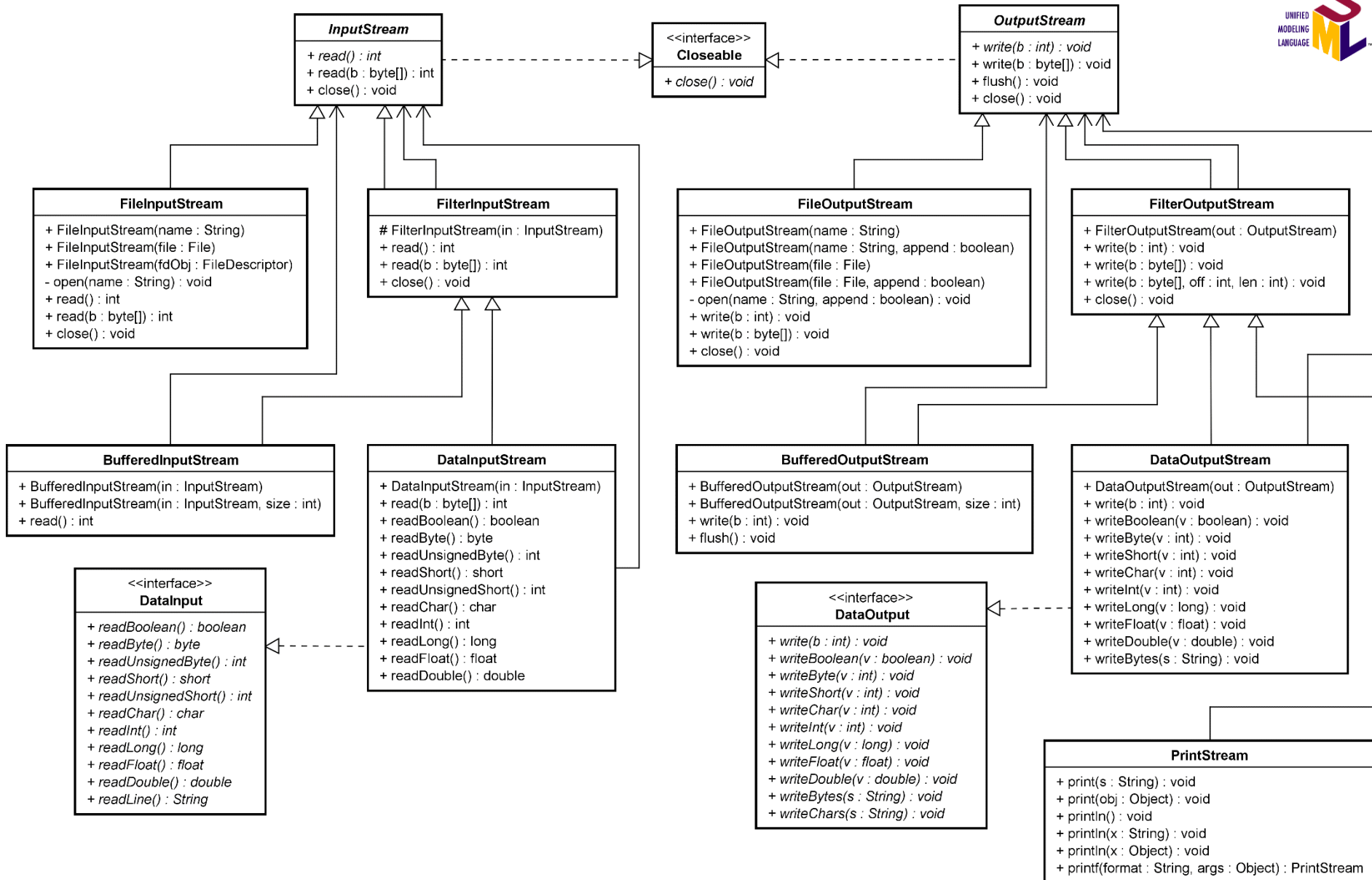
- **Achtung:** Textbasierende Datenströme sind → **encodiert!**

Byte Datenströme

Byte Datenströme - Grundlagen

- Basisklassen für alle Byte-Datenströme in Java: `InputStream` und `OutputStream`
- Die Daten werden Byte-weise behandelt (Werte `0..255`), in Java aber als `int` dargestellt.
 - Ende eines Streams (`EOS`) bzw. einer Datei (`EOF`) wird mit dem Wert `-1` signalisiert.
- Die vorinstanzierten Objekte `System.in` (Typ: `InputStream`) und `System.out` (Typ: `PrintStream`) für den Standard-I/O (`stdin/stdout/stderr`) bedienen sich dieser Byte-Datenströme.
- Das native Schreiben von einzelnen Datenwerten oder auch ganzer Objekte (→ Objekt-Serialisierung) basiert auf Byte-Datenströmen.

Byte-Datenströme: Ausgewählte Klassen



Byte-Datenströme – Beispiele von Klassen

- Regel: Alle I/O-Klassen, welche mit Byte-Datenströmen umgehen, tragen die Teilwörter **InputStream** bzw. **OutputStream** im Namen.
- Kurzbeschreibung einzelner Klassen:
 - **FileInputStream** und **FileOutputStream** unterstützen den I/O mit Dateien.
 - **BufferedInputStream** und **BufferedOutputStream** unterstützen **gepufferte** (cache) I/O-Streams.
 - **DataInputStream** und **DataOutputStream** unterstützen das lesen und schreiben von **elementaren** Datentypen.
 - **PrintStream** ermöglicht die Ausgabe von ASCII-Text (nicht Unicode!) direkt auf die Konsole (das ist **System.out**!).

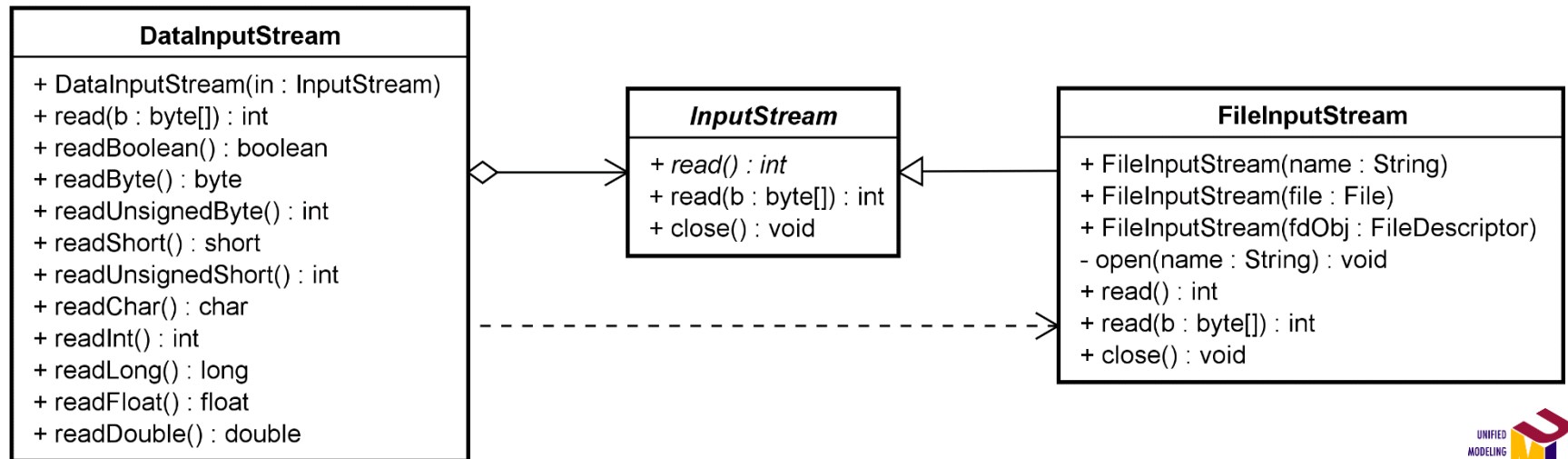
Kaskadierung von Stream-Objekten

- Beachten Sie, dass die meisten I/O-Klassen, welche von den Basisklassen **InputStream** bzw. **OutputStream** spezialisiert sind, diese Typen auch wieder verwenden und als formalen Parameter des Konstruktors entgegen nehmen.
- Somit lassen sich die verschiedenen Spezialisierungen zu «Ketten» verbinden (→ **kaskadieren**) und die einzelnen Funktionalitäten miteinander kombinieren!
 - Decorator- (bzw. Wrapper-) Pattern nach → GoF
(Input folgt im Modul VSK)
- Beispiel: **DataInputStream** lässt sich so z.B. wahlweise mit einer Datei oder einem TCP/IP-Socket verknüpfen, zusätzlich kann dieser Stream dann noch gepuffert (**Buffered***) sein oder nicht.

Byte-Datenstrom Beispiel: Einlesen eines double-Wertes

- Das folgende Codefragment öffnet die (Binär-)Datei `c:\demo.dat` und liest daraus einen `double`-Wert:

```
final FileInputStream fis =  
    new FileInputStream("c:\\demo.dat");  
final DataInputStream dis = new DataInputStream(fis);  
final double value = dis.readDouble();  
dis.close();
```

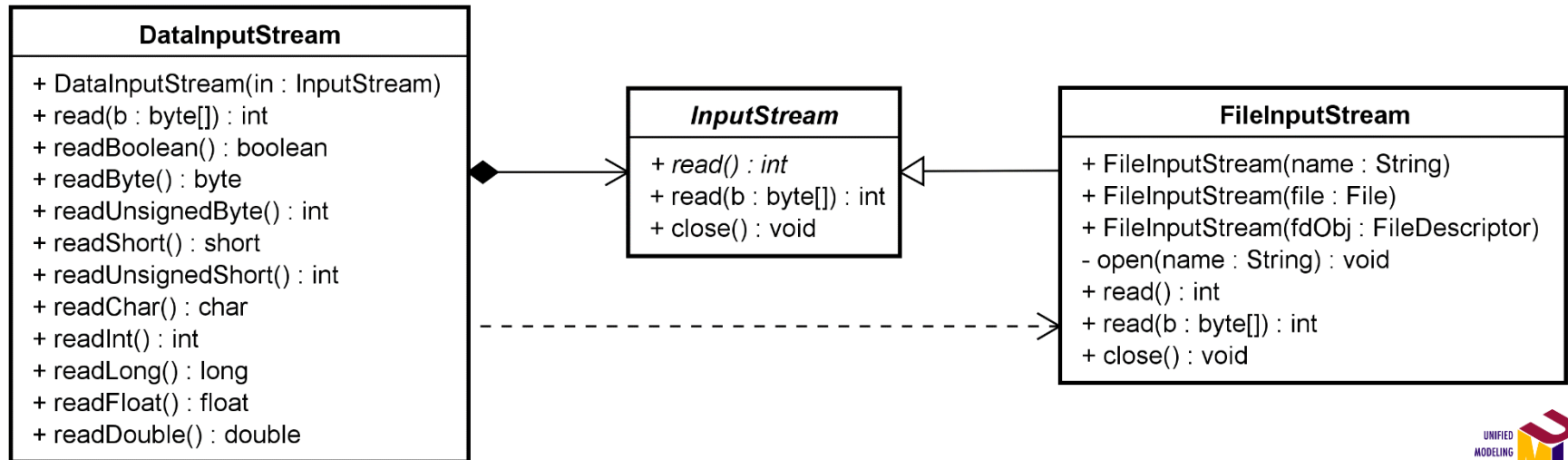


- Hinweis: **Exceptionhandling** aus Platzgründen entfernt!

Byte-Datenstrom Beispiel: Vereinfacht mit Komposition

- Ein `FileInputStream` wird dafür mit einem `DataInputStream` verknüpft (dekoriert), was **vereinfacht** auch so erfolgen kann:

```
final DataInputStream dis = new DataInputStream(  
    new FileInputStream("c:\\demo.dat"));  
final double value = dis.readDouble();  
dis.close();
```



- Hinweis: ➔ **Exceptionhandling** aus Platzgründen entfernt!

Exceptionhandling - Erweiterung

Exceptionhandling bei I/O-Streams

- Wenn wir I/O-Operationen durchführen, verwenden wird implizit Ressourcen, welche **nicht** unter der Kontrolle von **Java** (sondern des Betriebssystems) stehen, und welche darum **explizit** geöffnet **und geschlossen** werden müssen.
 - Öffnen: Erfolgt durch das Erzeugen (**new**) des entsprechenden Objektes in Java wiederum implizit.
 - Schliessen: Garbage Collection von Java ist dafür **nicht** geeignet (käme viel zu spät) → darum müssen wir I/O-Ressourcen immer **explizit** mit **close()** schliessen!
- Häufig hat man das in einem **finally**-Block gemacht (wird immer ausgeführt, egal was passiert!)
 - Aber was, wenn beim **close()** ein Fehler (Exception) passiert?
- Java 7 hat dafür eine erweiterte **try**-Anweisung eingeführt!

Exception-Handling: try-with-resources

- Mit Hilfe der **try-with-resources**-Anweisung wird sichergestellt, dass alle «Closable»-Objekte (Interface) am Ende ihres Sichtbarkeitsbereichs **automatisch** geschlossen werden!
- Beispiel von oben:

```
try (DataInputStream dis = new DataInputStream(  
    new FileInputStream("c:\\demo.dat"))) {  
    final double value = dis.readDouble();  
} catch (IOException ioe) {  
    LOG.error("Datei nicht lesbar.", ioe);  
}
```

- Man beachte: Der Aufruf von `close()` ist damit hinfällig!
 - Ressourcen werden **automatisch** und korrekt geschlossen.
 - Mehrere Ressourcen mit Semikolon trennen (Anweisungsblock).

Exception-Handling: Mehrere **catch**-Blöcke

- Besonders beim Fehlerhandling von I/O-Operationen können unterschiedlichste Fehler auftreten, welche zwar einzeln «gefangen» werden müssen, häufig aber auf identische Art behandelt werden (können).
- Dadurch entstehen sehr schnell unschöne → Coderedundanzen:

```
try {  
    ...  
} catch (SQLException sqlex) {  
    LOG.error("Fehler aufgetreten.", sqlex);  
} catch (SocketException soex) {  
    LOG.error("Fehler aufgetreten.", soex);  
} catch (IOException ioex) {  
    LOG.error("Fehler aufgetreten.", ioex);  
}
```



Hinweis: Der Text "Fehler aufgetreten." ist nicht aussagekräftig, hier nur aus Platzgründen verkürzt!

Exception-Handling: multiple catch

- Seit Java 7 kann man die Exception**typen** durch ein **pipe**-Symbol ("|", →oder) getrennt in einem einzigen **catch** auflisten.
- Beispiel (analog zu oben):



```
try {  
    ...  
} catch (IOException | SQLException | SocketException ex) {  
    LOG.error("Fehler aufgetreten.", ex);  
}
```

- Empfehlung: Dieses Konstrukt **nur** in Fällen benutzen, bei welchen **keine** (sinnvolle) gemeinsame Basisklasse existiert!
- Erinnerung: Die Klassen **Throwable**, **Error**, **Exception** und **RuntimeException** sind in aller Regel **keine** sinnvollen Basisklassen für **catch**-Statements!

Empfehlung - Erweitertes Exceptionhandling



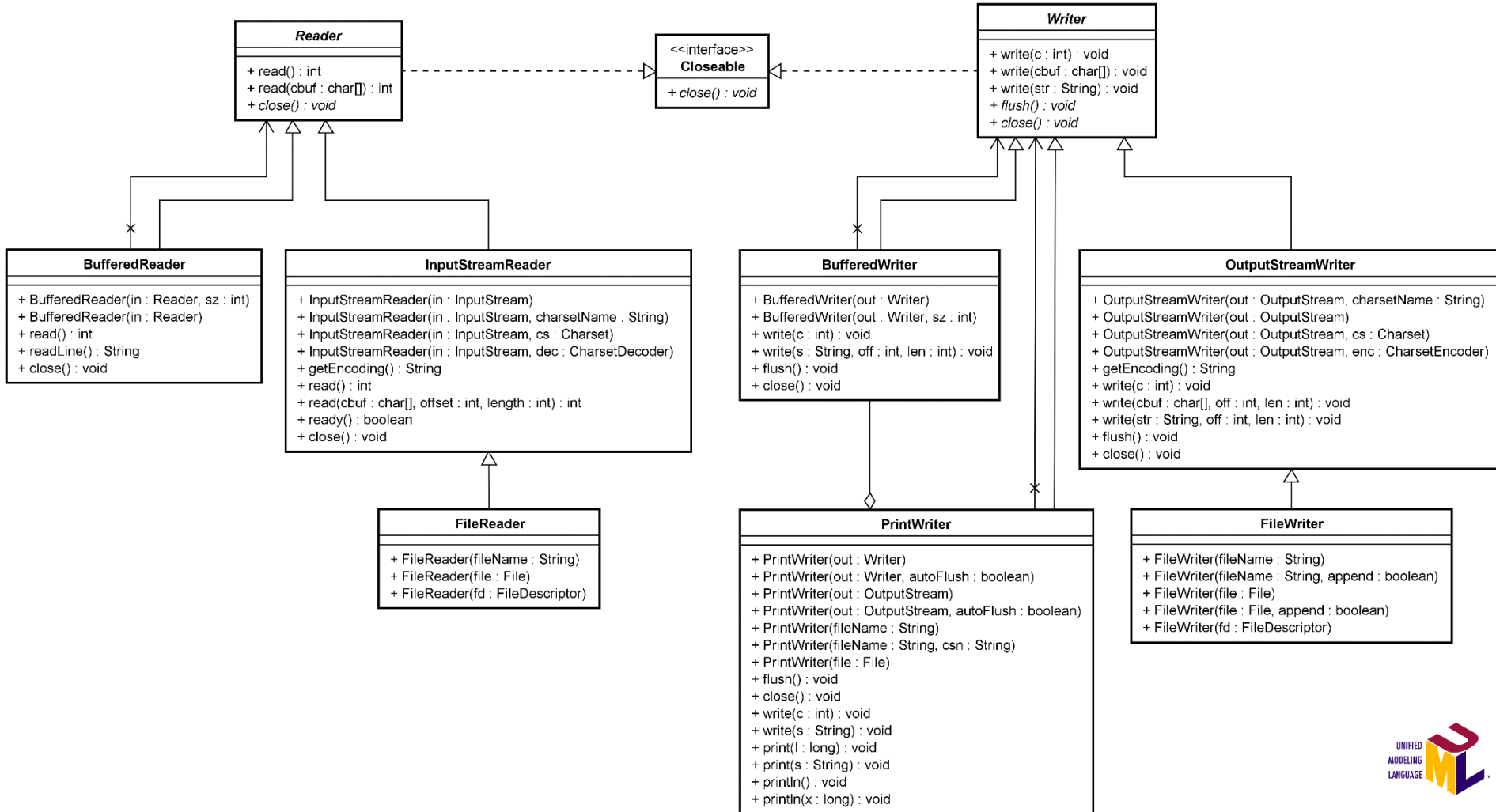
- **Wichtig:** Die zwei gezeigten Erweiterungen des Exception-Handlings sind **universell einsetzbar**, und somit **nicht spezifisch** auf I/O-Handling beschränkt!
- Vereinfachen den Code und reduzieren potenzielle Fehlerquellen.
- Beide Erweiterungen werden aber natürlich sehr oft im Zusammenhang mit I/O-Objekten (wg. des **Closable**-Interface) und entsprechenden Exceptions eingesetzt.

Zeichen-Datenströme

Zeichen-Datenströme - Grundlagen

- Basisklassen für alle Zeichen-Datenströme: **Reader** und **Writer**.
- Die Daten werden als 16bit-Unicode-Charakter behandelt (Werte **0..65535**), in Java aber als **int** dargestellt.
 - Ende eines Streams (**EOS**) bzw. einer Datei (**EOF**) wird mit dem Wert **-1** signalisiert.
- Mit Zeichen-Datenströmen können Text-Dateien (z.B. formatierter Text, XML, JSON → von Menschen lesbare Formate) sehr einfach gehandhabt werden.
 - Eine beliebig lange Kette von Zeichen, optional in Zeilen.
- **Achtung:** Zeichen-Datenströme sind → **encodiert!**
 - Java-intern werden alle Strings immer als **UTF-8** encodiert.

Zeichen-Datenströme: Ausgewählte Klassen



Zeichen-Datenströme: Beispiele von Klassen

- Regel: Alle IO-Klassen, welche mit Zeichen-Datenströmen arbeiten, tragen die Teilwörter **Reader** bzw. **Writer** im Namen.
- Kurzbeschreibung einzelner Klassen:
 - **Reader** und **Writer** unterstützen I/O mit Unicode-Zeichen.
 - **BufferedReader** und **BufferedWriter** erhöhen die Performance durch zeilenweise Pufferung der Lese- und Schreibzugriffe.
 - **InputStreamReader** und **OutputStreamWriter** sind **Adapterklassen** und ermöglichen die Kopplung von Byte-Datenströmen zu Zeichen-Datenströmen.
 - **FileReader** und **FileWriter** vereinfachen den Zugriff auf Text-Dateien, aber Achtung: Nutzen das Default-Encoding!
 - **PrintWriter** ermöglicht die Ausgabe von Unicode-Zeichen.

Zeichen-Datenströme: Schreiben in eine Textdatei

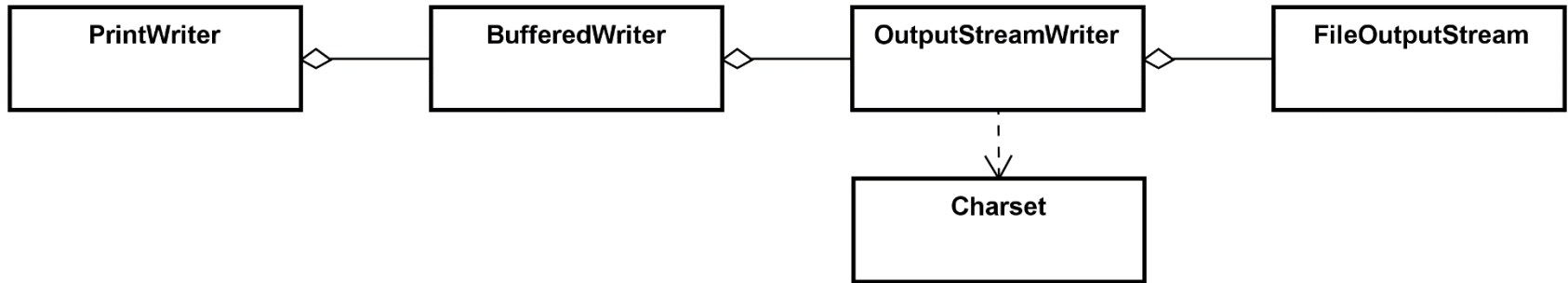
- Die folgende Methode schreibt zwei Zeilen in eine Datei:

```
public void writeTextFile(final String file) {  
    try (PrintWriter pw =  
        new PrintWriter(new BufferedWriter(new OutputStreamWriter(  
            new FileOutputStream(file), Charset.forName("UTF-8"))));  
        ) {  
        pw.println("Euro € und Umlaute äöü - alles da!");  
        pw.println("Plattformunabhängig fix mit UTF-8 encodiert!");  
        pw.flush();  
    } catch (IOException ioe) {  
        LOG.error(ioe.getMessage(), ioe);  
    }  
}
```

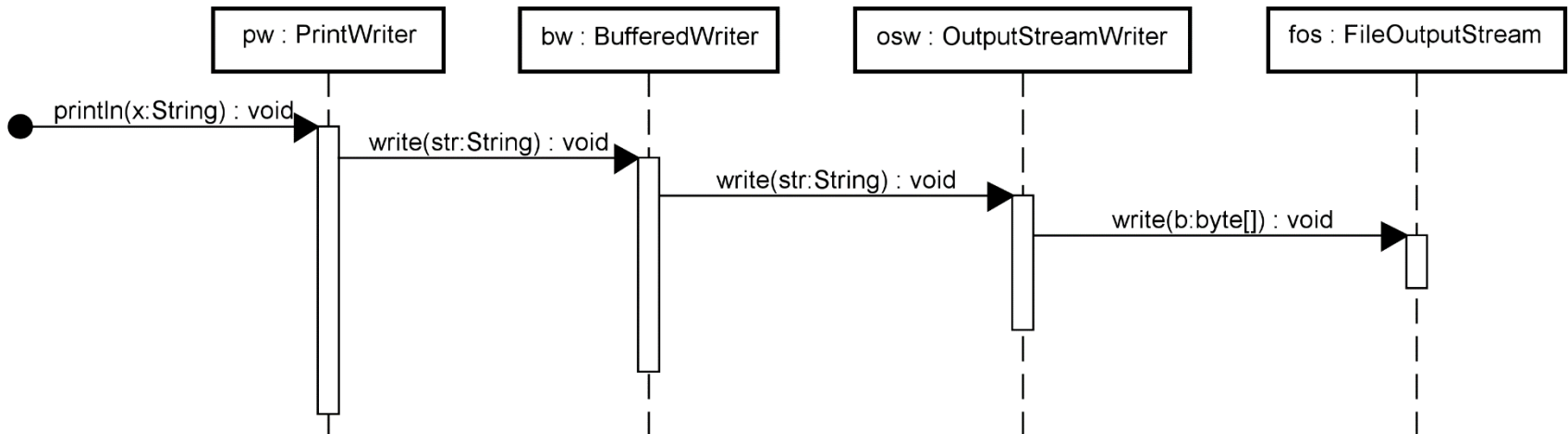
- Beispiel-Aufrufe dieser Methode
 - für Windows: `writeTextFile("C:\\demo.txt");`
 - für Linux: `writeTextFile("/tmp/demo.txt");`

Kaskadierung der Objekte und Delegation

■ Klassendiagramm (statisch Beziehungen):



■ Sequenzdiagramm (dynamischer Ablauf):



Zeichen-Datenströme: Lesen aus einer Textdatei

- Liest alle Zeilen einer Datei ein und gibt sie auf der Konsole aus:

```
public void readTextFile(final String file) {  
    if (new File(file).exists()) {  
        try (BufferedReader br =  
            new BufferedReader(new InputStreamReader(  
                new FileInputStream(file), Charset.forName("UTF-8"))))  
            ) {  
            String line;  
            while ((line = br.readLine()) != null) {  
                System.out.println(line);  
            }  
        } catch (IOException ioe) {  
            LOG.error(ioe.getMessage(), ioe);  
        }  
    }  
}
```

Euro € und Umlaute äöü - alles da!
Plattformunabhängig fix mit UTF-8 encodiert!

- Aufruf analog zu oben (wie beim Schreiben).

Zeichen-Datenströme: Warnung vor Default-Encoding

- Die folgenden stark vereinfachten Beispiele basieren auf dem Default-Encoding des jeweiligen Betriebssystems!

- Simpler Zeichen-Stream für Schreiben:

```
final Writer writer = new FileWriter(filename);
```

- Simpler Zeichen-Stream für Lesen:

```
final Reader reader = new FileReader(filename);
```

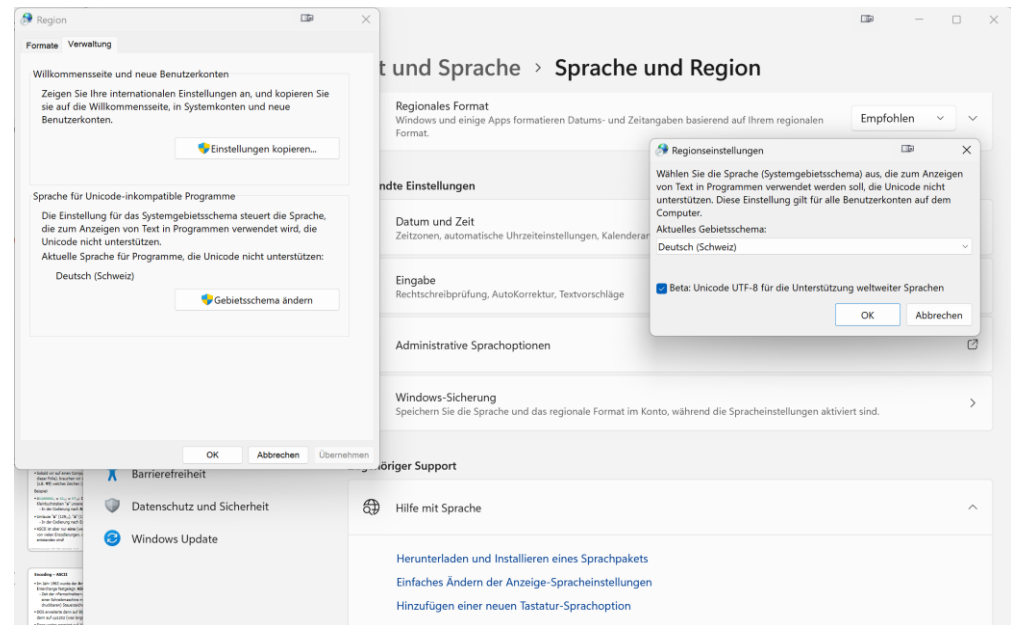


- Hinweis: Beide Beispiele können für eine Zeilen-Semantik noch mit einem **BufferedWriter** bzw. **~Reader** gewrapped werden.
- **Achtung:** Bei plattformübergreifendem Einsatz kommt es damit aber **zwangsläufig** zu Encoding-Problemen! Beispiel:
 - Linux schreibt, (UTF-8), altes Windows liest (cp1252).

➔ Darum verwenden wir für unseren Quellcode **immer** UTF-8!

Default Encoding von Betriebssystemen

- Linux-Betriebssysteme nutzen UTF-8 schon seit fast 20 Jahren in weiser Voraussicht als Default-Encoding.
 - Umstellung begann 2007 (Debian «Etch», April)
- Bei Microsoft Windows ist die Sache wesentlich intransparenter.
 - Ab Windows 10 (Version 1903 von 2019) wurden die Defaults verändert (von cp1252 auf UTF-8).
 - Noch immer als «Beta»-Feature in den Einstellungen vergraben.
 - Unterschiedlich, je nach Shell, und neues Terminal!



Encoding

Encoding - Grundlagen

- Sobald wir auf einen Computer Texte darstellen (z.B. der Text auf dieser Folie), brauchen wir eine Zuordnung, welche Zahlenwerte (z.B. 97) welches Zeichen (z.B. 'a') repräsentieren.

Beispiel:

- $01100001_2 = 61_{16} = 97_{10}$: Dieser Zahlenwert repräsentiert z.B. den Kleinbuchstaben "a" unseres Alphabetes.
 - In der Codierung nach **ASCII** (7bit).
- Umlaute "ü" (129_{10}), "ä" (132_{10}), "ö" (148_{10}) sind «Sonderzeichen».
 - In der Codierung nach Extended ASCII (8bit, >128).
- ASCII ist aber nur **eine** (wenn auch sehr grundlegende) **Variante** von vielen Encodierungen, die in den letzten Jahrzehnten entstanden sind!

Encoding – ASCII

- Im Jahr 1963 wurde der **American Standard Code for Information Interchange** festgelegt: **ASCII**
 - Zeit der «Fernschreiber», 7bit, im Wesentlichen die 95 Zeichen einer Schreibmaschine mit englischer Tastatur und 33 (nicht druckbaren) Steuerzeichen (z.B. Tab, CarriageReturn, LineFeed).
 - DOS erweiterte dann auf 8bit (Codepage **437**), Windows änderte dann auf **cp1252** (was lange für **de_CH** der Standard war).
 - Dann weiter normiert auf **ISO-8859-1 (latin-1)**, und später wiederum auf **ISO-8859-15 (latin-9)** erweitert.
 - Windows **11** (!) hat Codepage 65001 eingeführt:
Das «meiste» wird nun als UTF-8 gehandhabt!!!
- ➔ Das ist nur ein Teil der (halben) Wahrheit...



Encoding - Unicode



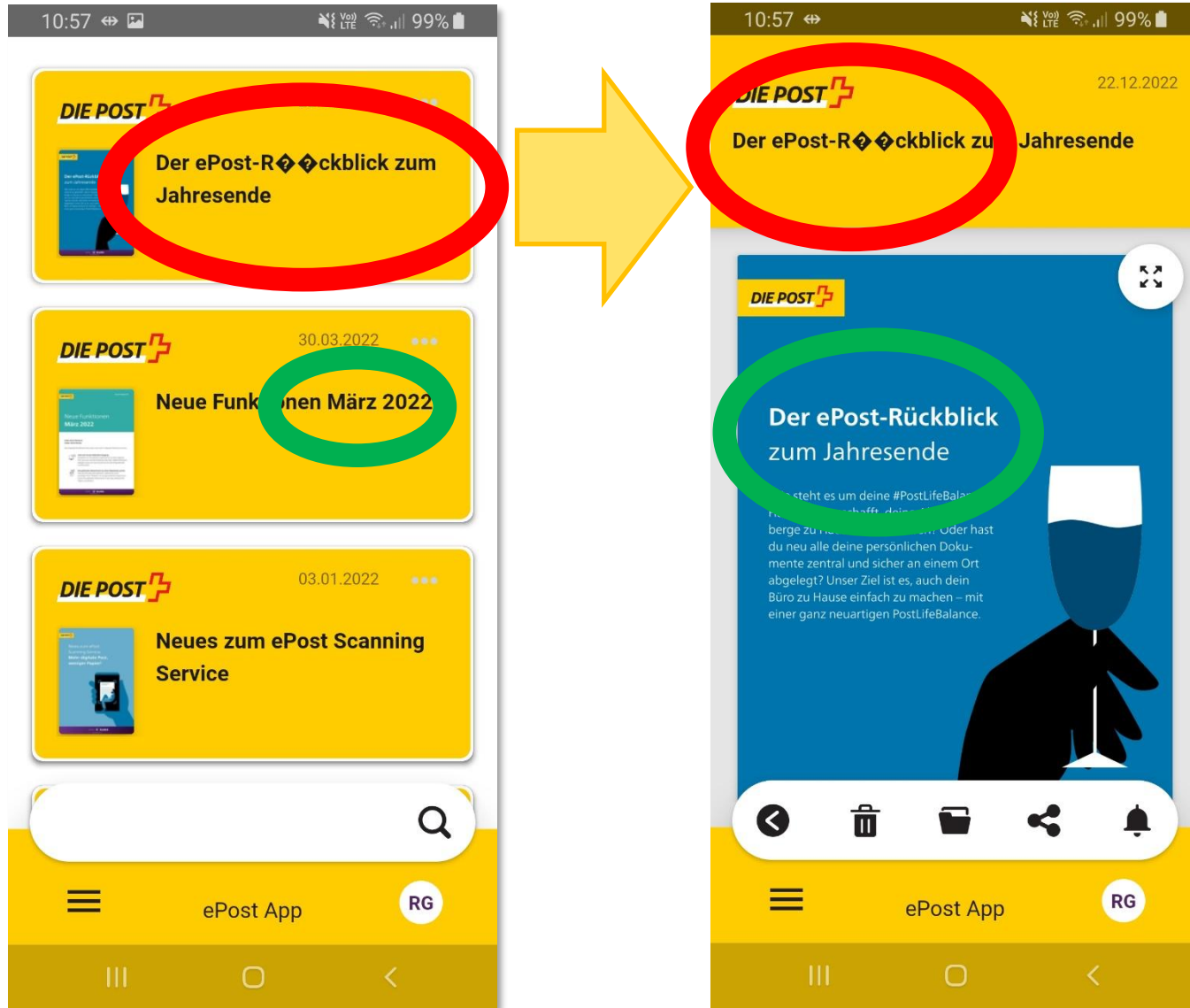
- Im Jahr 1991 entstand Unicode (Universal Coded).
 - Zeichensatz, welcher **alle** existierenden Zeichen darstellen kann.
 - Der Zeichensatz wird noch jetzt ständig erweitert (!).
 - Ursprünglich mit 16bit gestartet, wurde Unicode schon bald auf 32bit erweitert.
- Da damals Speicher noch nicht so günstig und in Unmengen verfügbar war wie heute, musste verhindert werden, dass sämtliche Dateien und Datenübermittlungen mit Unicode (16bit) plötzlich doppelt so gross wurden wie vorher mit ASCII (8bit).
- Zur Platzersparnis und zwecks Rückwärtskompatibilität (zumindest für Amerika) wurden spezielle Datenformate entwickelt:
Unicode Transformation Format (z.B. **UTF-8** oder **UTF-16**).

Beispiel: UTF-8 (Unicode Transformation Format)

Codierung von UTF8, leicht vereinfacht:

- **0xxx xxxx**
 - **ein** Byte, mit 7-bit Unicode → Direkt 7-bit-ASCII kompatibel!
- **110x xxxx** - **10xx xxxx**
 - **zwei** Bytes, mit maximal 11-bit grossem Unicode-Wert
- **1110 xxxx** - **10xx xxxx** - **10xx xxxx**
 - **drei** Bytes, mit maximal 16-bit grossem Unicode-Wert
- **1111 0xxx** - **10xx xxxx** - **10xx xxxx** - **10xx xxxx**
 - **vier** Bytes, mit maximal 21-bit grossem Unicode-Wert
- ...
- Details für Interessierte: <https://de.wikipedia.org/wiki/UTF-8>

Encoding – «The real world...» - 1



Schei 

Encoding

Encoding – **Wichtige** Hinweise und Empfehlungen



- Encoding-Probleme gehören zur Kategorie der Probleme, die man definitiv nicht haben will.
 - Es will sich in der Regel auch kaum jemand darum kümmern.
- Darum: Thematik erst recht **ernst** nehmen, speziell bei Daten- und Dateiaustausch über verschiedene Länder, Sprachregionen oder Plattformen hinweg!
 - Was heute quasi Standard ist (➔ Internet, Globalisierung).
- Encoding-Probleme **nie** leichtfertig ignorieren.
 - Je später man die Probleme angeht, umso schlimmer wird es!
- Spezielle Vorsicht auch bei (zentralen) Datenbeständen (z.B. Datenbanken), die aus verschiedenen Quellen gefüttert werden!

UTF-8 Everywhere - Manifest

- <https://utf8everywhere.org/>

UTF-8 EVERYWHERE

MANIFESTO

1. Purpose of this document

Our goal is to promote usage and support of the UTF-8 encoding and to convince that it should be the default choice of encoding for storing text strings in memory or on disk, for communication and all other uses. We believe that our approach improves performance, reduces complexity of software and helps prevent many Unicode-related bugs. We suggest that other encodings of Unicode (or text, in general) belong to rare edge-cases of optimization and should be avoided by mainstream users.

This document contains special characters. Without proper rendering support, you may see question marks, boxes, or other symbols.

In particular, we believe that the very popular UTF-16 encoding (often mistakenly referred to as ‘widechar’ or simply ‘Unicode’ in the Windows world) has no place in library APIs except for specialized text processing libraries, e.g. ICU.

This document also recommends choosing UTF-8 for internal string representation in Windows applications, despite the fact that this standard is less popular there, both due to historical reasons and the lack of native UTF-8 support by the API. We believe that, even on this platform, the following arguments outweigh the lack of native support. Also, we recommend forgetting forever what ‘ANSI codepages’ are and what they were used for. It is in the user’s bill of rights to mix any number of languages in any text string.

Dateihandling in Java

Dateihandling mit Java: Herausforderungen

- Java ist eine **plattformunabhängige** Sprache.
 - **W**rite **o**nce, **r**un **a**n anywhere (WORA) – **ohne** Neukompilierung ist das Programm auf **verschiedenen** Betriebssystemen (die jeweilige VM vorausgesetzt) direkt lauffähig.
- Dafür müssen aber **einige Unterschiede** berücksichtigt werden:
 - Dateisysteme mit unterschiedlichen Attributen.
 - Pfaddarstellung und Laufwerke (**C:\temp** vs. **/tmp**)
 - Unterschiedliche Byteorder (Little-Endian vs. Big-Endian)
 - Zeilenumbrüche: **CR+LF** vs. **LF+CR** vs. **LF**
 - Zeichensätze/Encodings: **ISO-8859-1**, **cp1252**, **UTF-8**,...
- Java stellt Zeichenketten (Strings) intern immer als **UTF-8** dar
 - Ist platzsparend und direkt ASCII(-7bit) kompatibel.
- **Achtung** bei Pfadangaben: "****" ist ein Escapezeichen → "****" (!)

Hilfsklassen für Datei-Handling

Klasse `java.io.File`

- Die Klasse `java.io.File` hat nicht direkt mit der Ein- und Ausgabe von Streams zu tun.
- **File** stellt plattformunabhängige Betriebssystemfunktionen im Zusammenhang mit Dateien und Verzeichnissen bereit.
- Oft muss man aber auch plattformspezifische Unterschiede berücksichtigen, z.B. unterschiedliche Path-Separatoren: `;` versus `:`



File
<u>+ separatorChar : char = fs.getSeparator()</u> <u>+ separator : String = ""+separatorChar</u> <u>+ pathSeparatorChar : char = fs.getPathSeparator()</u> <u>+ pathSeparator : String = ""+pathSeparatorChar</u>
+ File(pathname : String) + File(parent : String, child : String) + File(parent : File, child : String) + File(uri : URI) + getName() : String + getParent() : String + getParentFile() : File + getPath() : String + isAbsolute() : boolean + getAbsolutePath() : String + getAbsoluteFile() : File + getCanonicalPath() : String + getCanonicalFile() : File + toURL() : URL + toURI() : URI + canRead() : boolean + canWrite() : boolean + exists() : boolean + isDirectory() : boolean + isFile() : boolean + isHidden() : boolean + lastModified() : long + length() : long + createNewFile() : boolean + delete() : boolean + deleteOnExit() : void + list() : String[] + list(filter : FilenameFilter) : String[] + listFiles() : File[] + listFiles(filter : FilenameFilter) : File[] + listFiles(filter : FileFilter) : File[] + mkdir() : boolean + mkdirs() : boolean + renameTo(dest : File) : boolean + setLastModified(time : long) : boolean + setReadOnly() : boolean + setWritable(writable : boolean, ownerOnly : boolean) : boolean + setWritable(writable : boolean) : boolean + setReadable(readable : boolean, ownerOnly : boolean) : boolean + setReadable(readable : boolean) : boolean + setExecutable(executable : boolean, ownerOnly : boolean) : boolean + setExecutable(executable : boolean) : boolean + canExecute() : boolean + listRoots() : File[] + getTotalSpace() : long + getFreeSpace() : long + getUsableSpace() : long + createTempFile(prefix : String, suffix : String, directory : File) : File + createTempFile(prefix : String, suffix : String) : File + compareTo(pathname : File) : int + equals(obj : Object) : boolean + hashCode() : int + toString() : String + toPath() : Path

Klasse `java.nio.Files` und Interface `java.nio.Path`

- Hinweis: Mit Java 5 (und nochmal mit Java 7) wurden einige I/O-Klassen und Konzepte etwas «aufgefrischt» und erweitert.
 - Mehr «native» Funktionen, auch abhängig (!) vom Dateisystem.

```
<<interface>>
Path

+ getFileSystem() : FileSystem
+ isAbsolute() : boolean
+ getRoot() : Path
+ getFileName() : Path
+ getParent() : Path
+ getNameCount() : int
+ getName(index : int) : Path
+ subpath(beginIndex : int, endIndex : int) : Path
+ startsWith(other : Path) : boolean
+ startsWith(other : String) : boolean
+ endsWith(other : Path) : boolean
+ endsWith(other : String) : boolean
+ normalize() : Path
+ relativize(other : Path) : Path
+ toUri() : URI
+ toAbsolutePath() : Path
+ toRealPath(options : LinkOption) : Path
+ toFile() : File
+ iterator() : Iterator<Path>
+ compareTo(other : Path) : int
+ equals(other : Object) : boolean
+ hashCode() : int
+ toString() : String
```



```
Files

+ newInputStream(path : Path, options : OpenOption) : InputStream
+ newOutputStream(path : Path, options : OpenOption) : OutputStream
+ newDirectoryStream(dir : Path) : DirectoryStream<Path>
+ newDirectoryStream(dir : Path, glob : String) : DirectoryStream<Path>
+ newDirectoryStream(dir : Path, filter : Filter<Path>) : DirectoryStream<Path>
+ createFile(path : Path, attrs : FileAttribute<?>) : Path
+ createDirectory(dir : Path, attrs : FileAttribute<?>) : Path
+ createDirectories(dir : Path, attrs : FileAttribute<?>) : Path
+ createTempFile(dir : Path, prefix : String, suffix : String, attrs : FileAttribute<?>) : Path
+ createTempFile(prefix : String, suffix : String, attrs : FileAttribute<?>) : Path
+ createTempDirectory(dir : Path, prefix : String, attrs : FileAttribute<?>) : Path
+ createTempDirectory(prefix : String, attrs : FileAttribute<?>) : Path
+ createSymbolicLink(link : Path, target : Path, attrs : FileAttribute<?>) : Path
+ createLink(link : Path, existing : Path) : Path
+ delete(path : Path) : void
+ deleteIfExists(path : Path) : boolean
+ copy(source : Path, target : Path, options : CopyOption) : Path
+ move(source : Path, target : Path, options : CopyOption) : Path
+ readSymbolicLink(link : Path) : Path
+ getFileStore(path : Path) : FileStore
+ isSameFile(path : Path, path2 : Path) : boolean
+ isHidden(path : Path) : boolean
+ probeContentType(path : Path) : String
+ getFileAttributeView(path : Path, type : Class<V>, options : LinkOption) : V
+ readAttributes(path : Path, type : Class<A>, options : LinkOption) : A
+ setAttribute(path : Path, attribute : String, value : Object, options : LinkOption) : Path
+ getAttribute(path : Path, attribute : String, options : LinkOption) : Object
+ readAttributes(path : Path, attributes : String, options : LinkOption) : Map<String, Object>
+ getPosixFilePermissions(path : Path, options : LinkOption) : Set<PosixFilePermission>
+ setPosixFilePermissions(path : Path, perms : Set<PosixFilePermission>) : Path
+ getOwner(path : Path, options : LinkOption) : UserPrincipal
+ setOwner(path : Path, owner : UserPrincipal) : Path
+ isSymbolicLink(path : Path) : boolean
+ isDirectory(path : Path, options : LinkOption) : boolean
+ isRegularFile(path : Path, options : LinkOption) : boolean
+ getLastModifiedTime(path : Path, options : LinkOption) : FileTime
+ setLastModifiedTime(path : Path, time : FileTime) : Path
+ size(path : Path) : long
+ exists(path : Path, options : LinkOption) : boolean
+ notExists(path : Path, options : LinkOption) : boolean
+ isReadable(path : Path) : boolean
+ isWritable(path : Path) : boolean
+ isExecutable(path : Path) : boolean
+ newBufferedReader(path : Path, cs : Charset) : BufferedReader
+ newBufferedReader(path : Path) : BufferedReader
+ newBufferedWriter(path : Path, cs : Charset, options : OpenOption) : BufferedWriter
+ newBufferedWriter(path : Path, options : OpenOption) : BufferedWriter
+ copy(source : InputStream, sink : OutputStream) : long
+ copy(in : InputStream, target : Path, options : CopyOption) : long
+ copy(source : Path, out : OutputStream) : long
+ readAllLines(path : Path, cs : Charset) : List<String>
+ readAllLines(path : Path) : List<String>
+ write(path : Path, bytes : byte[], options : OpenOption) : Path
+ write(path : Path, lines : Iterable<CharSequence>, cs : Charset, options : OpenOption) : Path
+ write(path : Path, lines : Iterable<CharSequence>, options : OpenOption) : Path
```



Empfehlungen

Empfehlungen – Nutzung von I/O-Streams in Java



- Grundsätzliche Überlegung: Byte- oder Zeichenstrom verwenden?
 - Byte-Datenströme: Effizient und kompakt, aber binär.
 - Zeichen-Datenströme: Für Menschen lesbar, aber deutlich weniger effizient (Beispiele: XML, JSON oder HTML).
- Bei Zeichen-Datenströmen **unbedingt** das Encoding beachten!
 - Bei echten Multi-Plattform-Anwendungen sollte unbedingt ein **explizites** Encoding (**Charset**) eingesetzt werden.
- Sauberes Exceptionhandling ist speziell bei I/O-Streams wichtig.
 - **try-with-resources** und «multi-catch» als nützliche Vereinfachungen.
- I/O-Streams sind OS-Ressourcen, welche immer sauber geschlossen werden sollten: **close()** oder aber **Closable**.

Quizfrage

Zusammenfassung

- «Altes» und bewährtes Konzept der Datenströme.
 - Abstraktion zwischen wohin, woher und in welcher Form.
 - Decorator-Pattern zur Verkettung von Datenströmen.
- Byte-Datenströme: Reine Datenströme, Byte-orientiert, in der Regel sehr effizient und platzsparend.
- Zeichen-Datenströme: Für Menschen lesbar, auf Zeichensätzen basierend, abhängig von Encoding, benötigen typisch mehr Platz.
- Erweitertes Exceptionhandling (nicht spezifisch für Datenströme):
 - **try**-with-resources: Automatisches Schliessen aller Streams.
 - multi-catch: Verschiedene Exceptiontypen in nur einem **catch**-Block abfangen und behandeln.
- Wichtige Hilfsklassen für Dateihandling: **File**, **Path** und **Files**.



Fragen?

Fragen bitte im
ILIAS-Forum

